DOOLY: Configuration-Agnostic, Redundancy-Aware Profiling for LLM Inference

Joon Ha Kim (*UT Austin);* Geon-Woo Kim (*UT Austin*), Anoop Rachakonda (*UT Austin*), Daehyeok Kim (*UT Austin*)

Project Page

InfraAI TEXAS The University of Texas at Austin

This material is based upon work supported by the National Science Foundation under Grant No. 2403026

- LLM profiling is necessary for latency predictions used in request scheduling, benchmarking, and simulation.
- Today's profile methodologies rely on *static* and *manual* profiling methods for every serving configuration, which costs significant amounts of GPU-hours.
- **Dooly automatically detects common operations across different configurations, enabling *quick* and *accurate* profiling for LLM serving workloads**

LLM Serving Increasingly Relies on Latency Predictions

Configuration search
Which GPU, engine, and model to deploy?
e.g., Vidur, LLMServingSim

Instance placement
Which parallelism, how many instances, on which GPUs?
e.g., DistServe, Splitwise

Request routing
Which replica serves this request fastest?
e.g., llm-d

Per-operation latency model: $t = f(\text{operation, workload})$

Built by profiling: run every GPU operation across token counts, batch sizes, and context lengths

Problem: Profiling Every Serving Configuration is Expensive

1 New configurations need manual operation and input dimension discovery

Llama-3.1-8B
FlashInfer · Sarathi
QKVLinear
PagedAttention
…

= / ≠

Command-R7B
FlashAttention · SGLang
QKVLinear
RadixAttention ?
…

`RadixAttention (Q, K, V)`
Q: 60 4096
K: 60 1024

2 Every configuration is re-profiled from scratch

Llama-3.1-8B
FlashInfer · vLLM
QKVLinear
PagedAttention
…

=

Qwen3-8B
FlashInfer · vLLM
QKVLinear
PagedAttention
…

27.7 h
GPU-hours to profile 4 similar 7–8B models
on one engine + backend (1× A100)

Our Vision: Profile Each Operation Once, Reuse Across Configurations

Any configuration (H, S, M) → 1 **Discover operations** from a Kineto trace → **Seen before?**

- yes → reuse
- no → profile (2) → Shared latency DB

Challenges

1 Traces lack the meaning of input dims

`matmul` 60 4096 × 4096 6144

Sweep every dim
1 4096
1024 6144
✗ shape mismatch

Compare shapes
Llama 60 4096
Qwen3 62 4096
✗ looks different

2 Stateful operations can't run in isolation

`unified_attention`

✓ In trace	✗ Not in trace
`q: [60, 4096]`	KV cache
`k, v: [60,1024]`	Attn metadata

Replay fails: missing context

Idea 1: Label Every Input Dimension via Taint Propagation

1 Taint the sources

`ModelConfig` → M 4096, M 128

`SchedulerOutput` → NT 15, NR 3

vLLM entry points

M model config **NT** # tokens **NR** # requests

2 Propagate through one forward pass

`qkv_proj` [NT 15, M 6144]
↓
`view` [NT 15, M 48, M 128]
↓
`attention` [NT 15, M 32, M 128]

Labels drive profiling: **M** fixed · **NT / NR** swept · mixed (NT×M) recomputed

Idea 2: Isolate State-Dependent Operations via Hierarchical Resolution

1 Fall back to the parent module

`Attention`
`append` ✓ | `unified_attention` ✗
Error: *missing context* — ***fallback to parent***
`Attention` ✓
`append` | `unified_attention`
children absorbed by parent

2 Reuse the engine's own setup

Engine startup: KV cache
Every forward pass: Attn metadata
engine's own API `set_forward_context()`
`Attention` ✓
runs alone at any sweep point

Result: finest granularity that runs — operations where possible, modules where needed

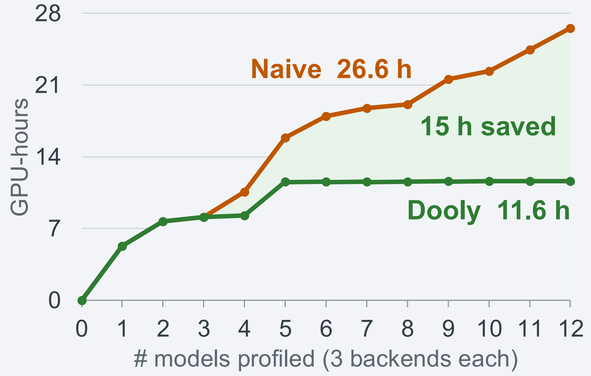
Idea 3: Detect Duplicate Operations via Signatures

1 **Operation + model dims**: `aten::linear` — ~~NT 60~~ M 4096 M 6144
+ 2 **GPU kernels**: FlashInfer / FlashAttention
+ 3 **Module settings**: `sliding_window` / `dtype = fp8`
→ SHA-256 `ba7816bf…` **DB key**

Llama-3.1-8B · attention → `ba7816bf…` ← **Qwen3-8B** · attention
✓ same key → reuse

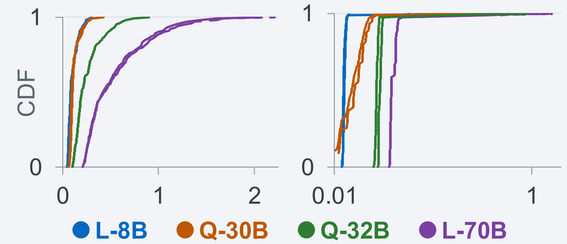
Results: Large Profile Savings and Accurate Configuration Modeling

56% fewer profiling GPU-hours



Simulation error vs. vLLM

< 5% TTFT **< 8%** TPOT



< 0.5% error in batching decisions